arXiv:2312.08688v2 [cs.CL] 15 Dec 2023

TigerBot: An Open Multilingual Multitask LLM

Ye Chen
Tiger Research
Shanghai, China
yechen@tigerbot.com

Wei Cai
Tiger Research
Shanghai, China
wei.cai@tigerbot.com

Liangmin Wu
Tiger Research
Shanghai, China
liangmin.wu@tigerbot.com

Xiaowei Li
Tiger Research
Shanghai, China
xiaowei.li@tigerbot.com

Zhanxuan Xin
Tiger Research
Shanghai, China
zhanxuan.xin@tigerbot.com

Cong Fu
Tiger Research
Shanghai, China
cong.fu@tigerbot.com

Abstract

We release and introduce the TigerBot family of large language models (LLMs) [1], consisting of base and chat models, sized from 7, 13, 70 and 180 billion parameters. We develop our models embarking from Llama-2 and BLOOM, and push the boundary further in data, training algorithm, infrastructure, and application tools. Our models yield meaningful performance gain over SOTA open-source models, e.g., Llama-2, specifically 6% gain in English and 20% gain in Chinese. TigerBot model family also achieves leading performance in major academic and industrial benchmarks and leaderboards [2]. We believe that TigerBot represents just a snapshot of lightning-fast progression in LLM open-source community. Therefore, we are thrilled to give back by publicly releasing our models and reporting our approach behind, with additional emphases on building SOTA LLMs in a democratized way and making LLMs of use in real-world applications.

1 Introduction

Large language models (LLMs) has shown unprecedented promise in a wide range of tasks. Since the phenomenal launch of ChatGPT, there have been breathtaking development in the community, mainly following three themes:

1. Fundamental capabilities, pushed forward by both proprietary models (GPT [4, 19], BARD [22], Claude [2]) and open-source models (BLOOM [27], Llama [32, 33]).
2. Computational economics, from data collection (e.g. Alpaca [31]), training (e.g. LoRA [11]), quantization (ExLlama [34]), and inference (e.g. TGI [12] and TensorRT [18]).
3. Application readiness, from APIs, plug-ins, function calling and agents, retrieval-augmented generation (RAG), long context window, to recently multimodality and role-playing.

The mainstream approach to building LLMs has been pretraining decoder-only transformers [35] on an extensive corpus of unsupervised textual data, followed by alignment with human preferences with labelled demonstration or comparison data, using supervised fine-tuning (SFT) or reinforcement learning with human feedback (RLHF). We have followed the same methodology, albeit made the following contributions:

[1] web: https://www.tigerbot.com/chat; github: https://github.com/TigerResearch/TigerBot

[2] As of this writing, TigerBot ranked top-tier open-source models in OpenCompass LLM Leaderboard , and CLiB Chinese LLM benchmark leaderboard.

Preprint. Under review.

1. A new training data mix with thorough experimental assessment and cleaning.
2. A stack of novel algorithmic and infrastructural implementations to make our models state-of-the-art (SOTA) in both performance and computational efficiency.
3. A thorough description of our implementations and observations from the field, in deploying our models to real-world applications, which help us prioritize research endeavors.

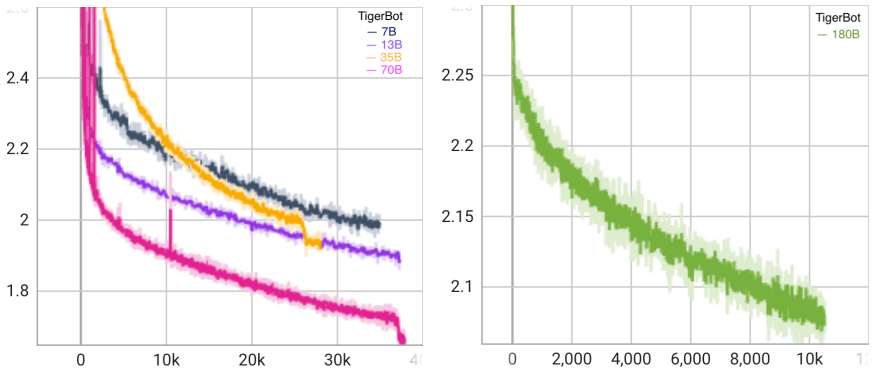
Besides achieving superior fundamental capabilities, we are dedicated to democratizing LLM development. To the best of our knowledge, TigerBot only incurs the least amount of computational costs (less than two million dollars over the time period April–December, 2023) and carbon footprint to produce probably one of the most comprehensive model families (from 7B to 180B, base and chat, with full stack of tools). This can only happen with an open-source spirit, hence we contribute a detailed elaboration on our methodology and experiences by the same token. Furthermore, we have taken measures to ensure the safety of our models.

2 TigerBot models

We are open-source releasing our models for free research and commercial use [3], as summarized in Table 1, along with a suite of developer tools. Figure 1 shows the training loss for pretraining.

Table 1: Tigerbot model family

Model	Base	Chat	API	Plug-in	Multi-modal	Context-length
7B	✓	✓	chat, fine-tune	search	image out	2k
13B	✓	✓	chat, fine-tune	search, doc	image in/out	32k
70B	✓	✓	chat, fine-tune	search, doc	image in/out	32k
180B	✓	✓	chat	search, doc	image in/out	2k



(a) Training loss for Tigerbot-70b etc. (b) Training loss for Tigerbot-180b

Figure 1: Training loss for Tigerbot models

Base model is a manifestation of world knowledge, serving as a foundation for downstream fine-tuning and applications. Chat model is fine-tuned to perform general-purpose tasks such as chat, question-answering (QA), generation, and so forth. API is a quick way to tap into TigerBot SOTA model capabilities from cloud with few lines of codes. Moreover, plug-in's allow developers and users to leverage the entire internet through modern search engines (search), or their own proprietary knowledge base (document).

2.1 Training data

In the pretraining stage, our training data consists of about 500 billion tokens, or 1.8TB plaintext data, which in turn was cleaned, deduped and down-sampled from 5.6TB data. Our data are chosen from

[3] TigerBot is released under Apache-2.0 license. However, since we continual pretrained from Llama-2 (7, 13, 70B) and BLOOM (180B), one shall consult with their open-source licenses respectively.

25 public and proprietary datasets, based on the following design considerations: (1) good quality, in terms of factuality, diversity, and format, based on our months' iterations and user feedback; (2) multilingual coverage, especially Chinese (e.g., WuDao and WanJuan corpus) and major eastern asian languages besides English (with zh:en roughly 5:5); and (3) multitask coverage, such as web (C4 and RefinedWeb), books (BookCorpus and Clibrary), wikipedia, codes (GitHub and Stack Overflow), academics (arXiv), and domain data (e.g., legal and patent). Table 2 shows our training data mixture and their sources, and Figure 2 illustrates proportions of datasets.

Table 2: Tigerbot training data

Dataset		Size (GB)	Tokens (B)	Source
Books	en-books	100.00	25.06	BookCorpus
	zh-books	154.00	39.23	Clibrary
	zh-textbook	2.20	0.74	WanJuan
WebTexts	en-c4	80.30	19.92	C4
	en-refinedweb	345.15	86.80	RefinedWeb
	en-webtext	39.00	10.14	OpenWebText
	zh-news	121.00	27.38	Tigerbot and WanJuan
	zh-webtext	614.00	147.59	WuDao and WanJuan
Papers	en-arxiv	38.00	12.52	arXiv
Codes	en-github	117.13	42.84	Github
	en-stackoverflow	24.80	7.97	Stack Overflow
Wiki	en-wiki	21.00	6.68	English wikipedia
	zh-wiki	2.79	1.72	Chinese wikipedia
	zh-baike	87.50	23.00	Tigerbot and WuDao
	ja-wiki	6.80	2.00	Japanese wikipedia
	ko-wiki	1.50	0.53	Korean wikipedia
Domain	en-stackexchange	6.80	1.91	Stack Exchange
	zh-law	35.03	9.42	Tigerbot and WanJuan
	zh-patent	17.00	4.66	WanJuan
	zh-sentiment	0.02	0.01	Cantonese sentiment
	Total	**1,814.02**	**470.12** [4]	

In the alignment learning stage, we collected 5 million instruction completion data for SFT, and 15k human annotated comparison data for RLHF, where 10k examples for rejection sampling and 5k for direct preference optimization (DPO). Our fine-tuning training data was chosen from about 30 open and proprietary datasets, and months of human annotation from real user prompts. The data mix is designed to represent a broad range of general-purpose tasks (e.g., generation, QA, and brainstorming), following a similar category in [19] and we further expand to about 100 subcategories (e.g., extract–from text to json, classification–sentiment classification).

We believe that data quality has a pivotal role in model quality, hence we have taken a systematic approach to ensuring data quality. Data from web is usually skewed in quality, even more so for corpus outside English. On the other hand, the mechanism underlying LLMs makes models have good memory. Our experiments find that less than a dozen low-quality examples would have the model learn and generate suboptimal results. Examples of low-quality data include: casual or oral language on the web, jargons in social media, ads, harmful contents, messy format and style, and so forth. We developed a set of rules and models to filter out about 10% low-quality data as follows. Given the large volume of the data, we designed the approach to have a complexity of $\mathcal{O}(n^2)$.

1. Filter out ill-format data using a set of rules, e.g., too many symbols or digits in prompts.
2. Dedup using exact string match.
3. Dedup using sequence simhash + longest common substring.

[4] later adding 5% holistic training data to make ∼500B tokens.

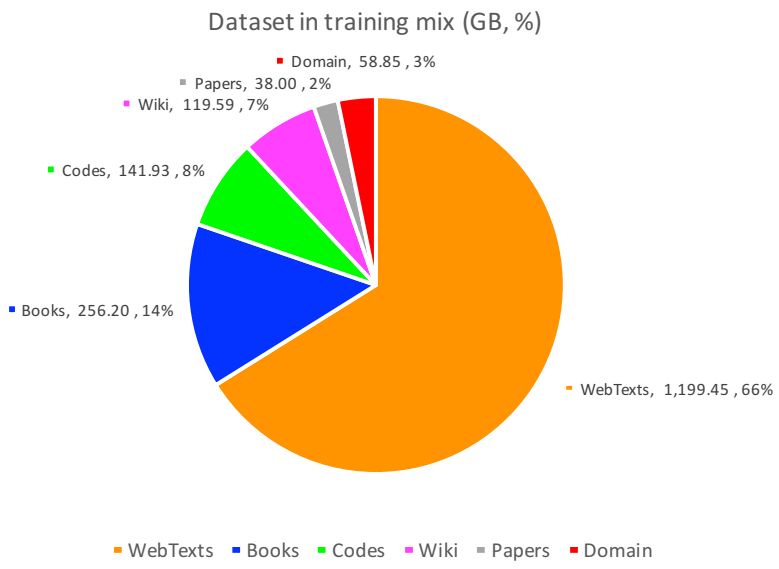


Figure 2: Training data proportions

4. Filter out harmful content using an ensemble of three SOTA safety models and a dictionary of sensitive words.

2.2 Training method

We innovate on the shoulders of our precedents, to fully inherit the data, computational resources, and intellectuality laid upfront. Models with 7, 13, and 70B parameters were continual pretrained from Llama-2 respective variants, whereas 180B was pretrained from BLOOM. We adopt most of the training setting and model architecture from our precedents, mainly decoder-only transformers [35], RoPE [29] and ALiBi [23] positional embedding, SwiGLU [28] and GeLU activation functions, respectively. We made further contributions as elaborated below. Our design objectives are: (1) to put forward a SOTA training infrastructure that can yield superior models in a computational economic manner, (2) multilingual coverage especially for Chinese, and (3) instrumental for application development.

Tokenizer Llama models have been lacking language representation other than English (e.g., Chinese only accounts for 0.13% in their training data), hence we expand tokenizer vocabulary in Tigerbot. First, we sample a 100GB corpus of Chinese and major eastern asian languages (mainly Japanese and Korean) from our pretraining data. Second, we train a Byte-Pair Encoding (BPE) SentenePiece tokenizer [10] using this non-English data to ensure representativeness in final tokenizer. Third, we merge the non-English tokenizer with the original Llama-2 one to make our final tokenizer [8]. The original vocabulary size is 32k, we expand to near but not exceeding 65k, to avoid doubling storage and IO in downstream tokenized binarized data. We also found that more than 100GB training data is unnecessary, the character coverage is almost identical; but the peak CPU memory would exceed 2TB, which is beyond mainstream hardware. For our 180B-parameter model, we keep the vocabulary size same as 250k, since BLOOM already has good multilingual coverage.

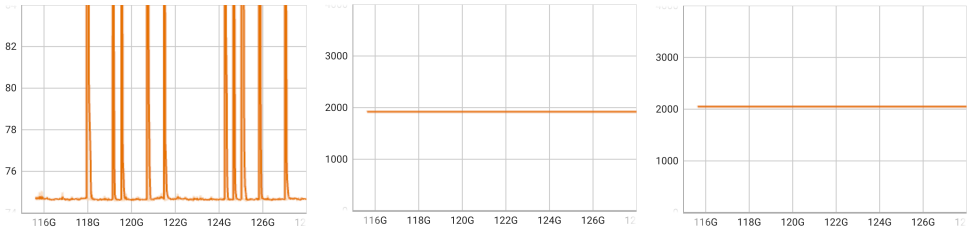
Training framework TigerBot model family has been trained using our private fork of Megatron-DeepSpeed [16], which implements 3D parallelism by combining ZeRO sharding, data parallelism (DP) and pipeline parallelism (PP) from DeepSpeed [25] with tensor parallelism (TP) from Megatron-LM [17]. Our fork made several upgrades as follows:

1. Bring Megatron-DeepSpeed Modeling class up to speed with SOTA architectural ingredients including CoreAttention, SwiGLU, grouped-query attention (GQA) [1], RoPE [29], and flash attention [9] adapted to Llama-2 architecture.

2. Design a better yet simple algorithm for pipeline partition. Given that a model with N attention blocks is divided into M stages, first $N \ mod \ M$ stages contain $\lceil N/M \rceil$ blocks and remaining blocks each has $\lfloor N/M \rfloor$ blocks. Compared to the original implementation where total stage is limited to several special numbers, our method is more flexible to alleviate the problem of skewed partition.
3. A set of scripts converting Megatron-DeepSpeed sharded weights to transformers weights and vice versa.

Tensor parallelism is particularly critical for training models over 100 billion parameters, since model size over 70B cannot fit into a single GPU while CPU offloading is slow and we want to avoid. On the other hand, more TPs introduce heavier communication and TP across nodes is impractical due to huge inter-node communication. Given a fixed value of TP $\times$ PP $\times$ DP, empirically we found smaller TP yields better global efficiency, largely because of heavier communications among tensor partitions (parallelizing matrix multiplication) than those of pipeline partitions (layers). Along together with techniques of GQA, flash attention, gradient accumulation and checkpointing, we have been able to find optimal configurations for different model sizes under various cluster resources. A back-of-the-envelop calculation follows.

Llama-2-13B pretrain GPU hours is 368,640 with 2TB tokens data, per the Llama-2 paper [33], thus we have: training-tokens/gpu-sec $= 1,507$. On the other hand, TigerBot-13B training throughput reaches 25.7 examples/sec or equivalently 74.6 sec/iteration, as shown in Figure 3, on a $32\times$ A100-40G GPU cluster, with a sequence length of $2,048$. Only after a preliminary geometric search of aforementioned parallelism configurations, we found an optimal setting is: TP=2, PP=8, DP=2, per-device-batch-size=2, and global-batch-size=1,920, to reach about 4M tokens global batch. Our efficiency reads: training-tokens/gpu-sec $= 1,645$ (109% of Llama-2 training). Also considering that Llama-2 used higher-end Meta's Research Super Cluster (A100-80G, 2TB CPU memory, RDMA inter-connection) [20], we believe that TigerBot's codebase has reached cutting-edge computational economics world wide.



(a) iteration time vs. tokens (b) batch size vs. tokens (c) seqlen vs. tokens

Figure 3: Training efficiency for Tigerbot models

Holistic training In the pretraining stage, we mix in 2-5% (of pretraining data) instruction completion data, preprocessed into an unsupervised format, e.g., `{instruction}+"\n"+{response}`, and remove duplicates from the original pretraining data that overlaps with SFT data knowledge. The rationale behind is: instruction completion (e.g., question-answering, generation) is essentially still a kind of human language continuation. We want models to learn some patterns of instruction following, holistically along with base knowledge during pretraining. The incurred additional computational cost is negligible, but the advantages are two folds:

1. Base models exhibit strong capability to follow instructions, right away before alignment. We performed a quick evaluation on SQuAD2.0 benchmark, and found that Tigerbot-13b-base reached 86% of the next token prediction accuracy as Tigerbot-13b-chat.
2. Since foundational capabilities (knowledge and instruction following) has been learned during pretraining, alignment learning can be lightweight. This further benefits rapid and economic application deployment in various verticals. Our experiments show that loss reaches 95% of convergence after one million examples of SFT training.

Supervised fine-tuning (SFT) The models are only trained on the response portion of supervised training data, using largely the same training routine, except data preprocessing. We first introduce

a pair of special tokens to markup instruction and response, respectively, for each example. We then process data examples individually (truncate or pad to maximum sequence length) or grouped (concatenate into maximum sequence length) into trainer. For the grouped method, we implement attention mask such that attention would not be computed cross examples. SFT data is usually quite sparse (few long examples and most within `max-seq-length`). The grouped method gives 5 to 10$\times$ speedup, albeit the individual way should yield higher accuracy intuitively. However, our experiments show that there is no noticeable performance difference between two methods, we then choose the grouped method in our production runs. We also notice that grouping method may introduce some degree of robustness and generality into the model, analogous to that human also learn from noises.

Reinforcement learning with human feedback (RLHF) In the RLHF stage, we first adopt rejection-sampling fine-tune with human-in-the-loop as follows:

1. Sample 10k prompts from SFT data and real user interactions.
2. Generate 10 random responses (e.g., with temperature=0.6) for each prompt, using our best candidate chat model.
3. Rank the generations, of all prompts 90% using reward model (a 13B model fine-tuned for text classification), 5% using human ranking, and 5% using human editing (gold set).
4. Mix the top-ranked generations into SFT data and perform fine-tuning.
5. Iterate the above process on a monthly basis, as we collect real user prompts.

We further apply the direct preference optimization (DPO) algorithm [24] on top of the rejection-sampling fine-tuned weights, using 5k gold comparison data. We choose DPO for it is simple to implement, efficient to train, while performing as well as or better than PPO-based methods. By intrinsically treating the LLM to be fit as a reward model, DPO essentially formulates a classification problem for pairwise comparison, with a simple cross-entropy loss as follows [24]:

$$\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta\log\frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right)\right] \tag{1}$$

where σ is a logistic function, and β is a hyperparameter that controls the deviation from the reference policy π_{ref}. Also as in [24], empirically we found DPO is more stable to train, likely due to its elegant formulation and does not need separate networks as in actor-critic style PPO algorithms. Efficient and stable training leads to rapid iteration in application deployment.

Long sequence Long sequence capability of a model tends to be important in applications, e.g., long context window to read a novel, or long generation length to write a book, all at once. Since text data follows a power law, i.e., most is short and few is long, inference with long sequence can be analogous to magnifying a picture. There are two key factors: (1) the resolution of the original image, and (2) the interpolation algorithm. The former is the length of training samples and the latter is RoPE extrapolation method.

During training TigerBot, we increase the RoPE base frequency to 500k [37] and group training samples to 4k. Attention parallelism could also be done along the sequence length dimension to allow training length exceed the limit of total GPU memory on a single node. This may yield extra-long sequence, e.g., several hundred thousand for certain applications, but at a higher cost of speed. Tensor parallelism between nodes becomes extremely slow due to huge communication, which indirectly limits the sample length in each node. We choose not to mix in special-purpose long-sequence data to preserve the generality of our models, but in the alignment stage we observed about 2‰ examples exceeding 2k tokens.

During the inference phase, length extrapolation is achieved through interpolating RoPE position embeddings [29]. The main difference among popular methods like Dynamic and YaRN [21] lies in their interpolation techniques. The challenge is how to maintain output consistency. In the implementations of Dynamic and YaRN by Transformers [13] and TGI [12], the approach involves "caching the position embeddings of the longest sequences seen". Under this implementation, even if the model observes the same input, the output may differ due to variations in the length of the cached position embeddings. Tigerbot addresses this by calculating the sum of the `input-token-length`

and `max-new-token-length` per request. This value is used as a reference when computing scaled position embeddings. This ensures the model's performance remains consistent when extrapolating lengths. The model's behavior for sequences not exceeding the training length is also preserved. We extrapolate the max sequence length to 32k using a RoPE scaling factor of 8.

Quantization Quantizing a LLM involves using a reduced-precision integer representation for weights and activations, which might be important for practical considerations of GPU memory limitation and fast inference. We implemented both static and dynamic quantizations.

In static quantization, the weights and activations of the model are computed using a calibration dataset in advanced. TigerBot models are quantized using ExLlamaV2 [34], which is based on the same optimization method as GPTQ. We demonstrated up to 3× speedup and 4× memory reduction for Tigerbot-4bit quantized models with negligible loss in accuracy.

In dynamic quantization, the weights are still quantized ahead of time post training, but the activations are quantized during inference on the fly. In particular, we use 8-bit weight and 16-bit activation quantization (W8A16). Our experiments show that 8-bit activation may incur significant accuracy degradation similar to [38], while W8A16 yields a good balance between accuracy and speedup. The dynamic approach has advantages in adapting to various serving hardwares, especially for those bottlenecked more by memory bandwidth than compute.

Safety We have performed safety filtering on the training data, we also take measures to mitigate safety risk during training and inference at runtime for any user-interfacing TigerBot products. We adopt a safety category consisting of 5 categories and 31 subcategories. Main categories include:

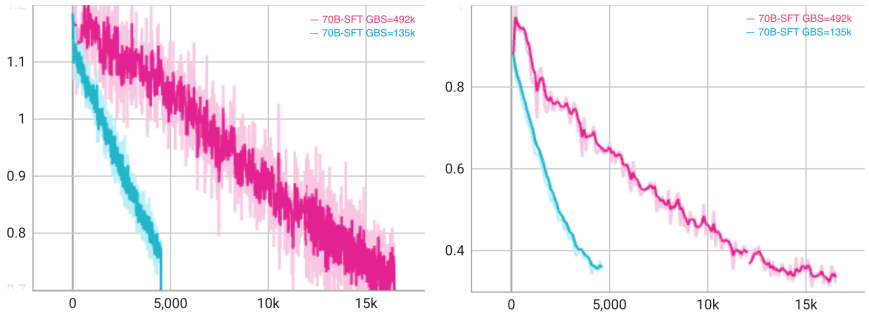
1. Violating core values of national and social security
2. Discriminatory content
3. Commercial illegal and unregulated activities
4. Infringement of others' legitimate rights and interests
5. Inability to meet the security requirements for some special-purpose services, e.g., medical information services and critical information infrastructure.

During training, we use human annotation to collect about 40k safety demonstration data, in consultation with administration guidelines and domain experts. This data is then fed into our alignment learning as well as pretraining per holistic training. The safety training data is refreshed on a monthly basis and reflected into our iterative alignment process. Both data and training level safety measures are preventive, while runtime-level safety check is protective.

During runtime inference, user input is examined safety first before feeding into model to generate. Would either user input or model generation be detected harmful, our products provide a default yet suggestive response to users. All contents undergo a two-stage examination, first a dictionary of about 120k sensitive vocabulary, followed by an ensemble of three BERT-based classifiers. These safety classifiers are trained on millions of annotated positive (violating) samples and focused on different aspects in the aforementioned safety categories. The dictionary is designed to be comprehensive to ensure high recall, whereas a good precision is achieved by tuning the positive threshold from safety classifiers. The final safety label is a parameterized function of dictionary detection and classifier output, and may well be varied for different domains and applications. Moreover, we have safety team to keep our dictionary and classifiers up to date with emerging topics and administration guidelines.

Hyperparameters We pretrained TigerBot models using a global batch size (GBS) of 4M tokens, while fine-tuned models with a GBS as small as 100–400k tokens. Our experiments show that, given high quality data, smaller batch for finer-grained updates can yield lower loss, as shown in Figure 4. We pretrained models for one epoch over training data, fine-tuned for two epochs, then followed by alignment learning for one epoch.

We used the adamW optimizer, with $\beta_1 = 0.9, \beta_2 = 0.95, \epsilon = 10^{-5}$. We applied a cosine decaying learning rate (LR) schedule between $[2.0^{-5}, 2.0^{-4}]$ for 7B and 13B, and $[1.0^{-5}, 1.0^{-4}]$ for 70B and 180B model pretraining. For fine-tuning, the LR is $[2.0^{-6}, 2.0^{-5}]$. We used warmup steps close to 1% of training steps, a weight decay rate of 0.1, and gradient clipping to 1.0. All trainings were performed under bfloat16 precision and flash attention, except for 180B we used gradient checkpointing instead.



(a) training loss (b) validation loss

Figure 4: Tigerbot-70b-SFT loss with different batch size

Training hardware Our training cluster consists of 512× A100-40G GPUs (64 nodes × 8 GPUs), equipped with NVLink intra-node GPU interconnects, and RoCE (RDMA over Converged Ethernet) inter-node communications. Each node has 64-core Intel Platinum CPU and 1024GB of RAM.

2.3 Evaluation

Evaluating LLMs is a challenging yet critical problem. It serves as a systematic basis for model selection, and reveals areas for improvements. However, performing evaluation only at the end of the training pipeline puts expensive computations at a risk of sinking, possibly just for a small negligence in data or bugs in codes. Moreover, developers have found that often there are discrepancies between benchmark metrics and human perception. To address these challenges, we develop a three-stage evaluation approach as below.

Table 3: Tigerbot base model evaluation results

Lang.	Task	Benchmark	TigerBot		Llama-2	
			70B-base	13B-base	70B-base	13B-base
En	Code	HumanEval	28.66	18.29	31.10	15.27
	Commonsense Reasoning	PIQA	83.30	79.33	82.21	79.21
		SIQA	48.77	48.52	46.01	46.32
		HellaSwag	78.62	72.64	79.46	74.96
		WinoGande	69.38	64.17	69.69	64.09
		OpenBookQ	88.60	72.40	57.40	57.00
	Reading Comprehension	BoolQ	67.52	63.18	69.69	71.50
	Math	GSM8K	65.66	35.86	63.99	28.81
	Multi-choice Questions	MMLU	68.68	55.35	69.58	55.81
	Average (En)		**66.58**	**56.64**	**63.24**	**54.77**
Zh	Reading Comprehension	CMRC	85.93	66.57	68.97	76.73
		C3	77.37	67.01	60.16	47.51
	Natural Language Inference	OCNLI	30.00	30.60	30.03	30.00
	Multi-choice Questions	C-EVAL	67.75	48.46	49.90	38.67
	Average (Zh)		**65.26**	**53.16**	**52.27**	**48.23**

Table 4: Tigerbot chat model evaluation results

Lang.	Task	Benchmark	TigerBot		Llama-2	
			70B-chat	13B-chat	70B-chat	13B-chat
En	Code	HumanEval	31.10	26.83	26.22	11.59
	Commonsense Reasoning	PIQA	83.57	80.09	80.25	78.67
		SIQA	51.89	49.44	51.59	50.97
		HellaSwag	76.68	70.63	77.63	74.71
		WinoGande	67.01	63.30	68.11	65.82
		OpenBookQ	85.00	67.40	85.00	80.00
	Reading Comprehension	BoolQ	80.67	78.87	78.00	78.32
	Math	GSM8K	84.91	51.25	58.91	54.62
	Multi-choice Questions	MMLU	68.03	55.94	64.84	54.61
	Average (En)		**69.87**	**60.42**	**65.62**	**59.43**
Zh	Reading Comprehension	CMRC	85.37	76.17	80.06	74.62
		C3	75.34	69.42	54.85	51.01
	Natural Language Inference	OCNLI	38.07	40.17	36.23	30.00
	Multi-choice Questions	C-EVAL	60.40	48.89	44.17	39.22
	Average (Zh)		**64.80**	**58.66**	**53.83**	**48.71**

1. During training for major checkpoints, we perform lightweight evaluations for a quick preview. We first draw a random sample of 100k train and validation examples from 10 major benchmarks including ARC [6], CommonsenseQA [30], SQuAD 2.0 [26], WebQuestions [3], and so forth. We then implemented a next-token prediction accuracy in Transformers' trainer, and a run of evaluation only takes several minutes, on one node, even for 70B and 180B models.

2. After training, we conduct a comprehensive evaluation on 13 mainstream benchmarks, covering 8 tasks and languages mainly of English and Chinese. The evaluation datasets are designed to cover a broad range of tasks such as math, reasoning, codes, and reading comprehension, and so forth. We report the results for both base and chat models in Tables 3 and 4, respectively. Base models are tested using 0-shot, and otherwise we follow the implementation of OpenCompass [7] to promote reproducibility.

3. Furthermore, we carry out human evaluations on our top candidates. Human evaluation is usually considered gold standard for assessing natural language generation systems. However, since LLMs can perform a very wide range of tasks, it is impractical to collect a comprehensive dataset to yield statistically significant results. The subjective biases among human annotators and the nontrivial cost if evaluating iteratively are also important considerations. We employ human evaluation as a gatekeeper, in conjunction with automated benchmark to select models. We first collect 5k gold set prompts, largely from real user questions and unseen from any upstream process. The gold set is sampled to cover a broad range of tasks as described in Section 2.1, yet reflect real user behaviors (tones, typos, oral vocabulary, etc.). We then ask our human annotators to rate on helpfulness and safety using a 1-5 Likert scale. We develop an evaluation guideline to train our annotators, which includes detailed rules and examples for various types of generations. e.g., factuality for objective questions, diversity for subjective generations. We occasionally found that human evaluation results were not aligned tightly with benchmark metrics, we choose production models as a holistic tradeoff, also considering model degradation, serendipity and downstream application desiderata.

3 Applications

In this section, we elaborate on our implementations of a spectrum of applications, some are tools and consumer products, and some are real world deployed applications.

Long-context question answering Many applications involve reading comprehension (summarization and question-answering) based on a long context, e.g., reading papers, studying laws, QA based on an in-house knowledge base, and so forth. Most knowledge-based QA systems are implemented as a two-stage retrieval-reader pipeline. A dense retrieval narrows the context down to a small subset of passages, followed by a LLM reader to generate answers based on the retrieved contexts [14]. This setup has two limitations: (1) it is not suitable for summarizing and inductive questions, where context should be comprehensive; and (2) a large portion of errors may already occur at the retrieval stage, due to practical reasons e.g., noisy and ill-format data.

As described in Section 2.2, we have extrapolated the context length to 32k tokens, or approximately 50k characters as in a 50-page pdf or word document (Tigerbot's tokenizer has a character-to-token compression rate of 1.5× for Chineses and 5× for English). This context window is big enough for most ad-hoc knowledge-based QA tasks, therefore we skip the dense retrieval part and take a one-stop approach as follows.

1. Segmentation: if input text exceeds the max input length (e.g., 32k), segment it into chunks of max length, using line breaks to preserve semantics.
2. Filtering: upon receiving a user query, zero-shot prompt the LLM as a binary classifier to filter out irrelevant segments, similar to the approach in [5]. We compose the prompt as: `C:{context}+"\n"+Q:{query}+"\n"+"Can the above Q be answered by C?"`
3. Generation: first generate a response by each candidate segment as intermediate results, from which then generate the final response using a prompt like: `{intermediate results}+"\n"+{query}`. This recursive approach has been used in summarization task and shown superior performance [36].

Recursive summarization Summarization has been one major NLP task, and now can be seamlessly solved by LLMs. To effectively handle extensively long texts, we employ a recursive summarization approach, similar to [36] but we solely rely on LLM. We first chunk the input text into smaller and manageable segments (within `max-input-length`), following natural semantic boundaries such as section endings and line breaks. We then summarize each segment independently. In the final step, we aggregate these individual summaries, to generate a comprehensive and cohesive final summary. Domain specifics and desired length can be naturally guided by prompts, e.g., with prompt: `"Summarize the above article into 200 words, preserving key financials."`

Function calling Natural language interface is attractive to many applications, where users give instructions in natural language and systems can understand and perform tasks otherwise requiring structured inputs. The intrinsic capability of natural language understanding of LLMs can be leveraged to extract structured data from natural language query, and then to perform downstream tasks, namely function calling. We design the function calling capabilities relying on the underlying TigerBot models, and as three steps as follows.

1. Extraction: given a function definition, the underlying LLM is prompted to extract function arguments from a user query. We compose the prompt as: `F:{func}+"\n"+Q:{query}+"\n"+"Extract args from Q per F."+"\n"+JSON:.`
2. Calling: we then call the target function with the arguments extracted to get a response. Functions can be in-house systems or third-party APIs from web, e.g., stock quotes and weather lookup APIs.
3. Generation: with the returned function response, we prompt the LLM again as: `{func response}+"\n"+{query}` to get the final answer in natural language.

The end-to-end performance of function calling largely relies on the LLM's capabilities in natural language understanding and structural extraction. For this purpose, we intentionally mixed a mild portion of general-purpose extraction data in pretraining and fine-tuning. We have observed quite

satisfactory performance for some basic function calling tasks, e.g., math calculation and stock quotes lookup. Therefore, we believe that with further fine-tuning on domain-specific extraction data, the function calling capabilities can be of use in many real world applications, particularly in place of those costly and complex legacy systems just for structural extraction.

Online search LLMs can be augmented by search, to get factoid and real time context, also to some degree to alleviate the hallucination problem. We implement search augmentation as follows.

1. Preprocess and search: we first preprocess user query to be suitable for modern search engines, e.g., removing oral interjections and time reference resolution, then issue the query to search engines to get results.
2. Quality filtering and parsing: we then filter the search results into top 1-3 candidates based on some quality and timeliness conditions, e.g., site quality and if results within a week present, remove those older than one month. We also parse relevant contents from those structured results, e.g., weather and stock prices.
3. Generation: we finally generate the response to users by prompting the underlying LLM with: `{top search results}+"\n"+{query}`.

Role playing Given its rich embodiment of general knowledge and conversational capability, NPC (non-player character) in RPG games can be equipped with LLM to become more entertaining. One common requirement from gaming applications is to make LLMs act as some roles, e.g., dialogues and personal memory. To develop a role-playing LLM, there are two design objectives: (1) to train a LLM with role-playing capability, and (2) to adapt the LLM into the context of the game. Meanwhile, developers usually want to keep the general capabilities of the LLM to make the NPC more humanlike. Furthermore, in practice the approach needs to be somewhat certain, lightweight, and scalable. Our approach to role-playing LLM combines fine-tuning and retrieval-augmented generation (RAG) as follows, and the process is illustrated in Figure 5. Our approach was inspired by [15], but we added a fine-tuning step to get a gamified foundation.

1. Fine-tuning: we continual fine-tune a TigerBot chat model on a dataset of general-purpose multi-round role-playing dialogues, e.g., acting as a hero in a novel. Fine-tuning can gauge the model to have role-playing capability, albeit is time consuming and stochastic in nature.
2. Extraction: given a novel or plot as the background context of a game, we extract dialogues between characters and summarize their profiles, both using a general-purpose TigerBot chat model. Extracted dialogues and profiles are fed into an embedding index as the game knowledge base, under a hierarchy beginning with "role".
3. Inference: during runtime inference, given a user's role and question, we first dense-retrieve reference dialogues and profiles from the knowledge base, and then use the above fine-tuned role-playing LLM to generate response. The prompt is composed as: `{role profiles}+"\n"+{reference dialogues}+"\n"+{question}`.

Our approach is designed such that it can be scaled to different games rapidly and economically. The role-playing capability is general and learned through fine-tuning, while knowledge base extraction is specific to game and fast. An example shown in Figure 6 is from a role-playing game developed within 2-3 days and took one node of 8$\times$ A100 GPUs.

Intelligent hardware Intelligent hardwares can be equipped with LLMs to first have natural language interfaces and potentially to orchestrate other applications using capabilities like function calling. In our practice at this stage, there are three typical requirements as follows.

1. Instantaneous response: intelligent hardwares, such as speakers, wearables, and electric vehicles, are mainly used on-the-go, thus require response time to be near real time. The system end-to-end response time usually cannot exceed 2s, while there are other processes involved including ASR, TTS, etc. To achieve fast response, we use streaming generation, along with our optimized inference engine TGI (vLLM and KV cache) as described in Section 2.2.
2. Frequently asked questions (FAQ): intelligent hardwares are purposed for specific scenarios. To get users familiar with the setting, LLMs need to be able to answer a set of FAQs, e.g.,

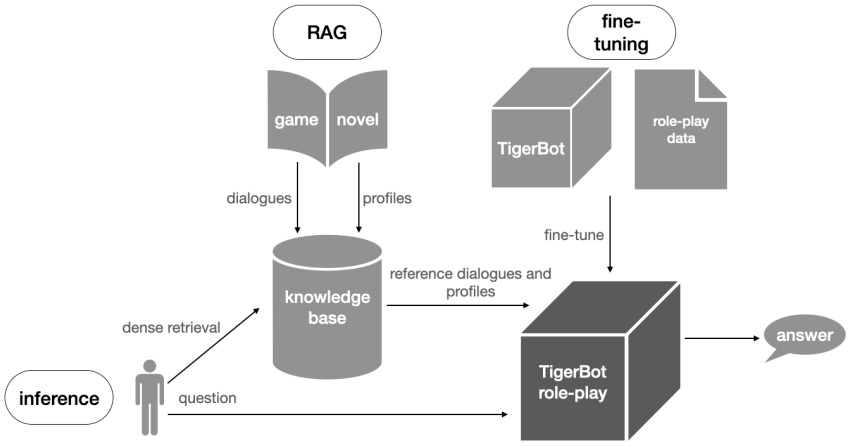


Figure 5: A scalable approach to Tigerbot's role-playing LLM

Figure 6: An example screenshot of a Tigerbot's role-playing with the novel "Genshin Impact"

What are the make and model of the car?. We first continual fine-tune a TigerBot chat model on domain data, e.g., user manual. Then during inference, we generate answers augmented by retrieval from a knowledge base filled with annotated FAQs. The knowledge base can be easily kept up to date.

3. Personification: hardwares become more vivid and entertaining would have been portrayed as a real-world character. Personification is manifested by a set of annotated data describing her profile (name and gender) and style (fun or professional). We then fine-tune a TigerBot chat model on the personification dataset for one to two epochs. To preserve the original general-purpose capabilities, the fine-tuning data is mixed with some general data. One rule of thumb for the mixture rate is that general data accounts for more than half.

4 Conclusion

In this work, we have introduced TigerBot, a family of pretrained and chat LLMs with parameter sizes of 7B to 180B. TigerBot has achieved SOTA performance with its high-quality training data and a stack of cutting-edge training methods and systems. We also placed an extra emphasis on practical applications, with our detailed implementations and observations for a spectrum of tools and real-world scenarios. We are strong believer of open innovations, and our work has benefited hugely from the LLM open-source community. Likewise, we hope that our work can contribute to the community for future theoretical and practical research and development.

The emergence of LLM has marked one of the most heart-bumping moments in decades of AI development history, largely in view of its overwhelming performance with extreme generality while being straightforward to build. We feel no less awe-inspiring. From our tremendous amount of experiments and implementations, however, we believe that we are still in the nascent stage of LLM and more broadly AGI evolution. There are challenges in reliable solutions to mission-critical tasks, sustainable infrastructure, disruptive not incremental user values from practical applications, to name just a few. The journey ahead is exciting yet equally arduous. Stay calm and happy coding.

References

[1] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv:2305.13245 [cs.CL]*, 05 2023.

[2] Anthropic. Claude 2. *https://www.anthropic.com/index/claude-2*, 06 2023.

[3] J. Berant, A. Chou, R. Frostig, and P. Liang. Semantic parsing on freebase from question-answer pairs. *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*, 2013.

[4] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei. Language models are few-shot learners. *arXiv:2005.14165v4 [cs.CL]*, 05 2020.

[5] H. Chen, R. Pasunuru, J. Weston, and A. Celikyilmaz. Walking down the memory maze: Beyond context limit through interactive reading. *arXiv:2310.05029 [cs.CL]*, 10 2023.

[6] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv:1803.05457 [cs.AI]*, 03 2018.

[7] O. Contributors. Opencompass: A universal evaluation platform for foundation models. *GitHub repository*, 2023.

[8] Y. Cui, Z. Yang, and X. Yao. Efficient and effective text encoding for chinese llama and alpaca. *arXiv:2304.08177 [cs.CL]*, 04 2023.

[9] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *arXiv:2205.14135 [cs.LG]*, 05 2022.

[10] Google. Sentencepiece. *GitHub repository*, 2023.

[11] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. Lora: Low-rank adaptation of large language models. *arXiv:2106.09685 [cs.CL]*, 06 2021.

[12] Huggingface. Text generation inference. *GitHub repository*, 2023.

[13] Huggingface. Transformers. *GitHub repository*, 2023.

[14] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. tau Yih. Dense passage retrieval for open-domain question answering. *EMNLP 2020*, 04 2020.

[15] C. Li, Z. Leng, C. Yan, J. Shen, H. Wang, W. MI, Y. Fei, X. Feng, S. Yan, H. Wang, L. Zhan, Y. Jia, P. Wu, and H. Sun. Chatharuhi: Reviving anime character in reality via large language model. *arXiv:2308.09597 [cs.CL]*, 2023.

[16] Microsoft. Megatron-deepspeed. *GitHub repository*, 2023.

[17] D. Narayanan, M. Shoeybi, J. Casper, P. LeGresley, M. Patwary, V. A. Korthikanti, D. Vainbrand, P. Kashinkunti, J. Bernauer, B. Catanzaro, A. Phanishayee, and M. Zaharia. Efficient large-scale language model training on gpu clusters using megatron-lm. *arXiv:2104.04473 [cs.CL]*, 04 2021.

[18] NVIDIA. Tensorrt open source software. *GitHub repository*, 2023.

[19] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, and R. Lowe. Training language models to follow instructions with human feedback. *arXiv:2203.02155v1 [cs.CL]*, 03 2022.

[20] O. Peckham. Meta completes research supercluster, announces next-gen datacenter. *HPCwire: https://www.hpcwire.com/2023/05/18/meta-completes-research-supercluster-announces-next-gen-datacenter/*, 05 2023.

[21] B. Peng, J. Quesnelle, H. Fan, and E. Shippole. Yarn: Efficient context window extension of large language models. *arXiv:2309.00071 [cs.CL]*, 09 2023.

[22] S. Pichai. An important next step on our ai journey. *https://blog.google/technology/ai/bard-google-ai-search-updates/*, 02 2023.

[23] O. Press, N. A. Smith, and M. Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. *arXiv:2108.12409 [cs.CL]*, 08 2021.

[24] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn. Direct preference optimization: Your language model is secretly a reward model. *arXiv:2305.18290 [cs.LG]*, 05 2023.

[25] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He. Zero: Memory optimizations toward training trillion parameter models. *arXiv:1910.02054 [cs.LG] and In Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '20)*, 10 2019.

[26] P. Rajpurkar, R. Jia, and P. Liang. Know what you don't know: Unanswerable questions for squad. *arXiv:1806.03822 [cs.CL]*, 06 2018.

[27] T. L. Scao, A. Fan, C. Akiki, E. Pavlick, S. Ilić, D. Hesslow, R. Castagné, A. S. Luccioni, F. Yvon, M. Gallé, J. Tow, A. M. Rush, S. Biderman, A. Webson, P. S. Ammanamanchi, T. Wang, B. Sagot, N. Muennighoff, A. V. del Moral, O. Ruwase, R. Bawden, S. Bekman, A. McMillan-Major, I. Beltagy, H. Nguyen, L. Saulnier, S. Tan, P. O. Suarez, V. Sanh, H. Laurençon, Y. Jernite, J. Launay, M. Mitchell, C. Raffel, A. Gokaslan, A. Simhi, A. Soroa, A. F. Aji, A. Alfassy, A. Rogers, A. K. Nitzav, C. Xu, C. Mou, C. Emezue, C. Klamm, C. Leong, D. van Strien, D. I. Adelani, D. Radev, E. G. Ponferrada, E. Levkovizh, E. Kim, E. B. Natan, F. D. Toni, G. Dupont, G. Kruszewski, G. Pistilli, H. Elsahar, H. Benyamina, H. Tran, I. Yu, I. Abdulmumin, I. Johnson, I. Gonzalez-Dios, J. de la Rosa, J. Chim, J. Dodge, J. Zhu, J. Chang, J. Frohberg, J. Tobing, J. Bhattacharjee, K. Almubarak, K. Chen, K. Lo, L. V. Werra, L. Weber, L. Phan, L. B. allal, L. Tanguy, M. Dey, M. R. Muñoz, M. Masoud, M. Grandury, M. Šaško, M. Huang, M. Coavoux, M. Singh, M. T.-J. Jiang, M. C. Vu, M. A. Jauhar, M. Ghaleb, N. Subramani, N. Kassner, N. Khamis, O. Nguyen, O. Espejel, O. de Gibert, P. Villegas, and et al. (293 additional authors not shown). Bloom: A 176b-parameter open-access multilingual language model. *arXiv:2211.05100 [cs.CL]*, 11 2022.

[28] N. Shazeer. Glu variants improve transformer. *arXiv:2002.05202 [cs.LG]*, 02 2020.

[29] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu. Roformer: Enhanced transformer with rotary position embedding. *arXiv:2104.09864 [cs.CL]*, 04 2021.

[30] A. Talmor, J. Herzig, N. Lourie, and J. Berant. Commonsenseqa: A question answering challenge targeting commonsense knowledge. *arXiv:1811.00937 [cs.CL]*, 11 2018.

[31] R. Taori, I. Gulrajani, T. Zhang, Y. Dubois, X. Li, C. Guestrin, P. Liang, and T. B. Hashimoto. Stanford alpaca: An instruction-following llama model. *GitHub repository*, 2023.

[32] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample. Llama: Open and efficient foundation language models. *arXiv:2302.13971 [cs.CL]*, 02 2023.

[33] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, D. Bikel, L. Blecher, C. C. Ferrer, M. Chen, G. Cucurull, D. Esiobu, J. Fernandes, J. Fu, W. Fu, B. Fuller, C. Gao, V. Goswami, N. Goyal, A. Hartshorn, S. Hosseini, R. Hou, H. Inan, M. Kardas, V. Kerkez, M. Khabsa, I. Kloumann, A. Korenev, P. S. Koura, M.-A. Lachaux, T. Lavril, J. Lee, D. Liskovich, Y. Lu, Y. Mao, X. Martinet, T. Mihaylov, P. Mishra, I. Molybog, Y. Nie, A. Poulton, J. Reizenstein, R. Rungta, K. Saladi, A. Schelten, R. Silva, E. M. Smith, R. Subramanian, X. E. Tan, B. Tang, R. Taylor, A. Williams, J. X. Kuan, P. Xu, Z. Yan, I. Zarov, Y. Zhang, A. Fan, M. Kambadur, S. Narang, A. Rodriguez, R. Stojnic, S. Edunov, and T. Scialom. Llama 2: Open foundation and fine-tuned chat models. *arXiv:2307.09288 [cs.CL]*, 07 2023.

[34] Turboderp. Exllamav2. *GitHub repository*, 2023.

[35] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. *31st Conference on Neural Information Processing Systems (NIPS 2017), Long Beach, CA, USA.*, 06 2017.

[36] J. Wu, L. Ouyang, D. M. Ziegler, N. Stiennon, R. Lowe, J. Leike, and P. Christiano. Recursively summarizing books with human feedback. *arXiv:2109.10862 [cs.CL]*, 09 2021.

[37] W. Xiong, J. Liu, I. Molybog, H. Zhang, P. Bhargava, R. Hou, L. Martin, R. Rungta, K. A. Sankararaman, B. Oguz, M. Khabsa, H. Fang, Y. Mehdad, S. Narang, K. Malik, A. Fan, S. Bhosale, S. Edunov, M. Lewis, S. Wang, and H. Ma. Effective long-context scaling of foundation models. *arXiv:2309.16039 [cs.CL]*, 09 2023.

[38] Z. Yao, R. Y. Aminabadi, M. Zhang, X. Wu, C. Li, and Y. He. Zeroquant: Efficient and affordable post-training quantization for large-scale transformers. *arXiv:2206.01861 [cs.CL]*, 06 2022.